

C++ Standard Library Issues to be moved in Toronto

Doc. no. P0698R0

Date: Revised 2017-06-19 at 02:20:00 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[2444](#). Inconsistent complexity for `std::sort_heap`

Section: 25.7.7.4 [sort.heap] **Status:** [Tentatively Ready](#) **Submitter:** François Dumont **Opened:** 2014-10-07 **Last modified:** 2017-05-31

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

While creating complexity tests for the GNU libstdc++ implementation [I stumbled](#) across a surprising requirement for the `std::sort_heap` algorithm.

In 25.7.7.4 [sort.heap] p3 the Standard states:

Complexity: At most $N \log(N)$ comparisons (where $N == \text{last} - \text{first}$).

As stated on the libstdc++ mailing list by Marc Glisse `sort_heap` can be implemented by N calls to `pop_heap`. As max number of comparisons of `pop_heap` is $2 * \log(N)$ then `sort_heap` max limit should be $2 * \log(1) + 2 * \log(2) + \dots + 2 * \log(N)$ that is to say $2 * \log(N!)$. In terms of $\log(N)$ we can also consider that this limit is also cap by $2 * N * \log(N)$ which is surely what the Standard wanted to set as a limit.

This is why I would like to propose to replace paragraph 3 by:

Complexity: At most $2N \log(N)$ comparisons (where $N == \text{last} - \text{first}$).

[2015-02 Cologne]

Marshall will research the maths and report back in Lenexa.

[2015-05-06 Lenexa]

STL: I dislike exact complexity requirements, they prevent one or two extra checks in debug mode. Would it be better to say $O(N \log(N))$ not at most?

[2017-03-04, Kona]

Move to Tentatively Ready. STL may write a paper (with Thomas & Robert) offering guidance about Big-O notation vs. exact requirements.

Proposed resolution:

This wording is relative to N3936.

1. In 25.7.7.4 [sort.heap] p3 the Standard states:

```
template<class RandomAccessIterator>
void sort_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
void sort_heap(RandomAccessIterator first, RandomAccessIterator last,
               Compare comp);
```

[...]

-3- Complexity: At most $2N \log(N)$ comparisons (where $N == \text{last} - \text{first}$).

2593. Moved-from state of Allocators

Section: 17.5.3.5 [allocator.requirements] **Status:** [Tentatively Ready](#) **Submitter:** David Krauss **Opened:** 2016-02-19 **Last modified:** 2017-06-15

Priority: 4

View other [active issues](#) in [allocator.requirements].

View all other [issues](#) in [allocator.requirements].

View all issues with [Tentatively Ready](#) status.

Discussion:

17.5.3.5 [allocator.requirements] suggests that the moved-from state of an allocator may be unequal to its previous state. Such a move constructor would break most container implementations, which move-construct the embedded allocator along with a compressed pair. Even if a moved-from container is empty, it should still subsequently allocate from the same resource pool as it did before.

```
std::vector<int, pool> a(500, my_pool);  
auto b = std::move(a); // b uses my_pool too.  
a.resize(500); // should still use my_pool.
```

[2016-02, Jacksonville]

Marshall will see if this can be resolved editorially.

After discussion, the editors and I decided that this could not be handled editorially. The bit about a moved-from state of an allocator being the same as the original state is a normative change. I submitted a [pull request](#) to handle the mismatched variables in the table.

Previous resolution [SUPERSEDED]:

This wording is relative to N4567.

1. Change 17.5.3.5 [allocator.requirements], Table 28 — "Allocator requirements" as indicated:

Note there's an editorial error in Table 28 in that line and the surrounding ones. The left column was apparently updated to use *u* and the right column is still using *a*/*a1*/*b*.

Table 28 — Allocator requirements

Expression	Return type	Assertion/note pre-/post-condition	Default
...			
X u(move(a)); X u = move(a);		Shall not exit via an exception. post: <u>u is equal to a and equal to the prior value of a</u> a1 equals the prior value of a.	
...			

[2016-06-20, Oulu, Daniel comments]

According to the current working draft, the situation has changed due to changes performed by the project editor, the revised resolution has been adjusted to N4594.

[2016-08 - Chicago]

Thurs AM: Moved to LEWG, as this decision (should allocators only be copyable, not movable) is design.

[2017-02 in Kona, LEWG responds]

Alisdair Meredith says that if you have a do-not-propagate-on-move-assignment, then the move of the allocator must compare equal to the original.

Have a data structure where all allocators are equal. Construct an element somewhere else moving from inside the container (which doesn't have a POCMA trait); you don't want the allocator of the moved-from element to now be different. So in that case, the allocator's move constructor must behave the same as copy.

We don't need to go as far as this issue, but going that far is ok for Bloomberg.

[2017-06-02 Issues Telecon]

We discussed containers that have sentinel nodes, etc, and so might have to allocate/deallocate using a moved-from allocator - and decided that we didn't want any part of that

Adjusted the wording slightly, and moved to Tentatively Ready

Previous resolution [SUPERSEDED]:

This wording is relative to N4594.

1. Change 17.5.3.5 [allocator.requirements], Table 28 — "Allocator requirements" as indicated:

Table 28 — Allocator requirements			
Expression	Return type	Assertion/note pre-/post-condition	Default
...			
X u(std::move(a)); X u = std::move(a);		Shall not exit via an exception. post: <u>u is equal to a and equal to the prior value of a</u> u is equal to the prior value of a.	
...			

Proposed resolution:

This wording is relative to N4594.

1. Change 17.5.3.5 [allocator.requirements], Table 28 — "Allocator requirements" as indicated:

Table 28 — Allocator requirements			
Expression	Return type	Assertion/note pre-/post-condition	Default
...			
X u(std::move(a)); X u = std::move(a);		Shall not exit via an exception. post: <u>The value of a is unchanged and is equal to u</u> u is equal to the prior value of a.	
...			

[2597](#). std::log misspecified for complex numbers

Section: 26.5.8 [complex.transcendentals] **Status:** [Tentatively Ready](#) **Submitter:** Thomas Koeppel **Opened:** 2016-03-01 **Last modified:** 2017-03-13

Priority: 3

View all other [issues](#) in [complex.transcendentals].

View all issues with [Tentatively Ready](#) status.

Discussion:

The current specification of std::log is inconsistent for complex numbers, specifically, the Returns clause (26.5.8 [complex.transcendentals]). On the one hand, it states that the imaginary part of the return value lies in the *closed* interval $[-i\pi, +i\pi]$.

On the other hand, it says that "the branch cuts are along the negative real axis" and "the imaginary part of $\log(x)$ is $+\pi$ when x is a negative real number".

The inconsistency lies in the difference between the mathematical concept of a branch cut and the nature of floating point numbers in C++. The corresponding specification in the C standard makes it clearer that if x is a real number, then $\log(x + 0i) = +\pi$, but $\log(x - 0i) = -\pi$, i.e. they consider positive and negative zero to represent the two different limits of approaching the branch cut from opposite directions. In other words, the term "negative real number" is misleading, and in fact there are *two distinct* real numbers, $x + 0i$ and $x - 0i$, that compare equal but whose logarithms differ by $2\pi i$.

The resolution should consist of two parts:

1. Double-check that our usage and definition of "branch cut" is sufficiently unambiguous. The C standard contains a lot more wording around this that we don't have in C++.
2. Change the Returns clause of `log` appropriately. For example: "When x is a negative real number, $\text{imag}(\log(x + 0i))$ is π , and $\text{imag}(\log(x - 0i))$ is $-\pi$."

Current implementations seem to behave as described in (2). ([Try-it-at-home link](#))

[2016-11-12, Issaquah]

Move to Open - Thomas to provide wording

[2016-11-15, Thomas comments and provides wording]

Following LWG discussion in Issaquah, I now propose to resolve this issue by removing the normative requirement on the function limits, and instead adding a note that the intention is to match the behaviour of C. This allows implementations to use the behaviour of C without having to specify what floating point numbers really are.

The change applies to both `std::log` and `std::sqrt`.

Updated try-at-home link, see [here](#).

[2017-03-04, Kona]

Minor wording update and status to Tentatively Ready.

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Change the "returns" element for `std::log` (26.5.8 [complex.transcendentals] p17):

```
template<class T> complex<T> log(const complex<T>& x);
```

-16- *Remarks*: The branch cuts are along the negative real axis.

-17- *Returns*: The complex natural (base- e) logarithm of x . For all x , $\text{imag}(\log(x))$ lies in the interval $[-\pi, \pi]$, and when x is a negative real number, $\text{imag}(\log(x))$ is π . [Note: The semantics of `std::log` are intended to be the same in C++ as they are for `clog` in C. — end note]

2. Change the "returns" element for `std::sqrt` (26.5.8 [complex.transcendentals] p25):

```
template<class T> complex<T> sqrt(const complex<T>& x);
```

-24- *Remarks*: The branch cuts are along the negative real axis.

-25- *Returns*: The complex square root of x , in the range of the right half-plane. If the argument is a negative real number, the value returned lies on the positive imaginary axis. [Note: The semantics of `std::sqrt` are intended to be the same in C++ as they are for `csqrt` in C. — end note]

Proposed resolution:

This wording is relative to N4606.

1. Change the "returns" element for `std::log` (26.5.8 [complex.transcendentals] p17):

```
template<class T> complex<T> log(const complex<T>& x);
```

-16- *Remarks*: The branch cuts are along the negative real axis.

-17- *Returns*: The complex natural (base- e) logarithm of x . For all x , $\text{imag}(\log(x))$ lies in the interval $[-\pi, \pi]$, and when x is a negative real number, $\text{imag}(\log(x))$ is π . [Note: the semantics of this function are intended to be the same in C++ as they are for `clog` in C. — end note]

2. Change the "returns" element for `std::sqrt` (26.5.8 [complex.transcendentals] p25):

```
template<class T> complex<T> sqrt(const complex<T>& x);
```

-24- *Remarks*: The branch cuts are along the negative real axis.

-25- *Returns*: The complex square root of x , in the range of the right half-plane. If the argument is a negative real number, the value returned lies on the positive imaginary axis. [Note: The semantics of this function are intended to be the same in C++ as they are for `csqrt` in C. — end note]

[2783](#). `stack::emplace()` and `queue::emplace()` should return `decltype(auto)`

Section: 23.6.4.1 [queue.defn], 23.6.6.1 [stack.defn] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-10-14 **Last modified:** 2017-03-13

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

The stack and queue adaptors are now defined as:

```
template <class... Args>
reference emplace(Args&&... args) { return c.emplace_back(std::forward<Args>(args)...); }
```

This breaks any code using `queue<UserDefinedSequence>` or `stack<UserDefinedSequence>` until the user-defined containers are updated to meet the new C++17 requirements.

If we defined them as returning `decltype(auto)` then we don't break any code. When used with `std::vector` or `std::deque` they will return reference, as required, but when used with C++14-conforming containers they will return void, as before.

[2016-11-12, Issaquah]

Sat AM: P2

[2017-03-04, Kona]

Status to Tentatively Ready.

Proposed resolution:

This wording is relative to N4606.

1. Change return type of `emplace` in class definition in 23.6.4.1 [queue.defn]:

```
template <class... Args>
reference decltype(auto) emplace(Args&&... args) { return c.emplace_back(std::forward<Args>(args)...); }
```

2. Change return type of `emplace` in class definition in 23.6.6.1 [stack.defn]:

```
template <class... Args>
reference decltype(auto) emplace(Args&&... args) { return c.emplace_back(std::forward<Args>(args)...); }
```

[2932](#). Constraints on parallel algorithm implementations are underspecified

Section: 25.4.3 [algorithms.parallel.exec] **Status:** [Ready](#) **Submitter:** Dietmar Kühl **Opened:** 2017-02-05 **Last modified:** 2017-03-13

Priority: Not Prioritized

View other [active issues](#) in [algorithms.parallel.exec].

View all other [issues](#) in [algorithms.parallel.exec].

Discussion:

Section 25.4.3 [algorithms.parallel.exec] specifies constraints a user of the parallel algorithms has to obey. Notably, it specifies in paragraph 3 that executions of element access functions are indeterminately sequenced with respect to each other. Correspondingly, it is the user's obligation to ensure that these calls do not introduce data races (this is also clarified in a note on this section).

Unfortunately, there is no constraint that, at least, mutating element access functions like `operator++()` on an iterator are called on different objects. An odd implementation of a parallel algorithm could increment a shared iterator from two threads without synchronisation of its own and the user would be obliged to make sure there is no data race!

For example:

```
template <typename FwdIt>
FwdIt adjacent_find(std::execution::parallel_policy, FwdIt it, FwdIt end)
{
    if (2 <= std::distance(it, end)) {
        FwdIt tmp(it);
        auto f1 = std::async([&tmp]() { ++tmp; });
        auto f2 = std::async([&tmp]() { ++tmp; });
        f1.get();
        f2.get();
    }
    return std::adjacent_find(it, end);
}
```

This code is, obviously, a contrived example but with the current specification a legal implementation of `adjacent_find()`. The problem is that, e.g., for pointers there is a data race when incrementing `tmp`, i.e., the function can't be used on pointers. I don't think any of the containers makes a guarantee that their iterators can be incremented without synchronisation, i.e., the standard library doesn't have any iterators which could be used with this algorithm!

Of course, no sane implementation would do anything like that. However, they are allowed to be implemented as above, making it unnecessarily hard and probably inefficient to use the algorithms: in portable code any user of the parallel algorithms needs to deal with the possibility that mutating operations on the same object are called from different threads. There is a constraint missing for the parallel algorithm implementations which limits calls to, at least, some element access functions to be applied only to different objects if there is synchronisation done by the algorithm. At least, obviously mutating operations like the iterator increment/decrement operators need to be correspondingly constrained.

On the other hand, it seems reasonable that a shared data structure stores, e.g., a predicate used concurrently by all threads. This use is quite reasonable and there is nothing wrong. That is, demanding that all element access functions are called on different objects between different threads would possibly adversely over-constraining the algorithms. Only the element access functions deliberately changing the object need to be constrained.

Based on checking all algorithms in the standard library taking an `ExecutionPolicy` template parameter there are broadly four groups of template parameters:

1. Parameters with a known set of possible arguments: `ExecutionPolicy` (execution policies listed in 20.19 [execpol]).
2. Parameters specifying types of objects which are expected not to change: `BinaryOperation`, `BinaryPredicate`, `Compare`, `Function`, `Predicate`, `UnaryFunction`, `UnaryOperation`, and `T` (all but the last one are function objects although I couldn't locate concepts for some of them — that may be a separate issue).
3. Parameters of mutable types which are also meant to be mutated: `InputIterator`, `OutputIterator`, `ForwardIterator`, `BidirectionalIterator`, `RandomAccessIterator`, and `Size` (the concept for `Size` also seems to be unspecified).
4. Some algorithms use `Generator` which seems to be a mutable function object. However, I couldn't locate a concept for this parameter.

The problematic group is 3 and possibly 4: mutations on the objects are expected. It seems the general approach of disallowing calling non-const functions without synchronisation applies. Note, however, that prohibiting calling of any non-const function from the algorithms would put undue burden on the implementation of algorithms: any of the accessor functions may be non-const although the concept assumes that the function would be const. The constraint should, thus, only apply to functions which may mutate the object according to their respective concept.

Suggested Resolution:

Add a statement prohibiting unsequenced calls to element access functions on the same object which are not applicable to const objects according to the corresponding concept. I'm not sure how to best specify the constraint in general, though.

Since the current algorithms use relatively few concepts there are fairly few operations actually affected. It may be reasonable at least for the initial version (and until we could refer to constructs in concepts in the language) to explicitly list the affected operations. I haven't done a full audit but iterator ++, --, @= (for @ being any of the operators which can be combined with an assignment), and assignments on all objects may be the set of affected element access functions whose use needs to be constrained.

Here is a concrete proposal for the change: In 25.4.2 [algorithms.parallel.user] add a paragraph:

Parallel algorithms are constrained when calling mutating element access functions without synchronisation: if any mutating element access function is called on an object there shall be no other unsynchronised accesses to this object. The mutating element access functions are those which are specified as mutating object in the concept, notably assignment on any object, operators ++, --, +=, and -= on any of the iterator or Size parameters, and any @= operators on the Size parameters.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4640](#).

1. Modify 25.4.2 [algorithms.parallel.user] as indicated:

-1- Function objects passed into parallel algorithms as objects of type Predicate, BinaryPredicate, Compare, and BinaryOperation shall not directly or indirectly modify objects via their arguments.

-?- Parallel algorithms are constrained when calling mutating element access functions without synchronisation: If any mutating element access function is called on an object there shall be no other unsynchronised accesses to this object. The mutating element access functions are those which are specified as mutating object in the concept, notably assignment on any object, operators ++, --, +=, and -= on any of the iterator or Size parameters, and any @= operators on the Size parameters.

[2017-03-03, Kona]

Dietmar provides improved wording. Issues with the PR before the change:

- The part before the colon is redundant: we don't need to state that.
- Replace "notably" with "specifically"
- swap() needs to be in the list.
- Not sure what "called on an object means"
- The assignment side is overconstrained: the right hand side is allowed.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4640](#).

1. Modify 25.4.2 [algorithms.parallel.user] as indicated:

-1- Function objects passed into parallel algorithms as objects of type Predicate, BinaryPredicate, Compare, and BinaryOperation shall not directly or indirectly modify objects via their arguments.

-?- If an object is mutated by an element access function the algorithm will perform no other unsynchronized accesses to that object. The mutating element access functions are those which are specified as mutating the object in the relevant concept, such as swap(.), ++, --, @=, and assignments. For the assignment and @= operators only the left argument is mutated.

[2017-03-03, Kona]

Dietmar finetunes wording after review by SG1.

[2017-03-03, Kona]

Move to Ready

Proposed resolution:

This wording is relative to [N4640](#).

1. Add a new paragraph following 25.4.3 [algorithms.parallel.exec] p1 as indicated:

-1- Parallel algorithms have template parameters named `ExecutionPolicy` (20.19) which describe the manner in which the execution of these algorithms may be parallelized and the manner in which they apply the element access functions.

-?- If an object is modified by an element access function the algorithm will perform no other unsynchronized accesses to that object. The modifying element access functions are those which are specified as modifying the object in the relevant concept [Note: For example, `swap()`, `++`, `--`, `@=`, and assignments modify the object. For the assignment and `@=` operators only the left argument is modified. — end note].

-2- [...]

[2937](#). Is `equivalent("existing_thing", "not_existing_thing")` an error?

Section: 27.10.15.12 [fs.op.equivalent] **Status:** [Tentatively Ready](#) **Submitter:** Billy Robert O'Neal III **Opened:** 2017-02-27 **Last modified:** 2017-03-13

Priority: 0

View all other [issues](#) in [fs.op.equivalent].

View all issues with [Tentatively Ready](#) status.

Discussion:

See discussion on the LWG mailing list with subject "Is `equivalent("existing_thing", "not_existing_thing")` an error?", abbreviated below:

Billy O'Neal:

The existing "an error is reported" effects say that an error is reported for `!exists(p1) && !exists(p2)`, but I'm not sure that treating `equivalent("existing_thing", "not_existing_thing")` as "false, no error" makes any more sense than for `equivalent("not_existing_thing", "not_existing_thing")`.

It's also unfortunate that the current spec requires reporting an error for `is_other(p1) && is_other(p2)` — there's no reason that you can't give a sane answer for paths to NT pipes. (Do POSIX FIFOs give garbage answers here?)

Davis Herring:

I'm fine with an error if either path does not exist. See also Late 29: I would much prefer

```
file_identity identity(const path&, bool resolve = true);
```

which would of course produce an error if the path did not exist (or, with the default `resolve`, was a broken symlink).

See Late 30 and 32 (31 has been resolved). FIFOs pose no trouble: you can even `fstat(2)` on the naked file descriptors produced by `pipe(2)`. (That said, I observe the strange inconsistency that Linux but not macOS gives both ends of a pipe the same `st_ino`.)

POSIX has no reason that I know of to treat any file type specially for `equivalent()`.

Billy O'Neal:

I think such a `file_identity` feature would be useful but we can always add it in addition to `equivalent` post-C++17.

Beman Dawes:

Looks good to me. Maybe submit this as an issue right away in the hopes it can go in C++17?

[2017-03-04, Kona]

Set priority to 0; Tentatively Ready

Proposed resolution:

This wording is relative to [N4640](#).

1. Make the following edits to 27.10.15.12 [fs.op.equivalent]:

```
bool equivalent(const path& p1, const path& p2);  
bool equivalent(const path& p1, const path& p2, error_code& ec) noexcept;
```

~~-1- Let s_1 and s_2 be file status determined as if by `status(p1)` and `status(p2)`, respectively.~~

~~-2- Effects: Determines s_1 and s_2 . If `(!exists(s1) && !exists(s2)) || (is_other(s1) && is_other(s2))`, an error is reported (27.10.7).~~

-3- Returns: true, if ~~$s_1 == s_2$~~ and p_1 and p_2 resolve to the same file system entity, else false. The signature with argument ec returns false if an error occurs.

-4- Two paths are considered to resolve to the same file system entity if two candidate entities reside on the same device at the same location. **[Note: On POSIX platforms, this is determined as if by the values of the POSIX stat structure, obtained as if by `stat()` for the two paths, having equal `st_dev` values and equal `st_ino` values. — end note]**

[-?- Remarks: `!exists(p1) || !exists(p2)` is an error.

-5- Throws: As specified in 27.10.7.

[2940](#). result_of specification also needs a little cleanup

Section: D.11 [depr.meta.types] Status: [Tentatively Ready](#) Submitter: Daniel Krüglér Opened: 2017-03-04 Last modified: 2017-06-15

Priority: 3

View all other [issues](#) in [depr.meta.types].

View all issues with [Tentatively Ready](#) status.

Discussion:

[P0604R0](#) missed a similar adjustment that was performed for the deprecated type trait templates `is_literal_type` and `is_literal_type_v` via LWG [2838](#):

Moving the `result_of` to Annex D means that the general prohibition against specializing type traits in [meta.type.synop]/1 does no longer exist, so should be explicitly spelled out. Wording will be provided after publication of the successor of [N4640](#).

[2017-03-04, Kona]

Set priority to 3

[2017-04-24, Daniel provides wording]

Instead of enumerating all the templates where adding a specialization is valid, a general exclusion rule is provided. Albeit currently not needed for the templates handled by this sub-clause, a potential exception case is constructed to ensure correctness for the first deprecated trait template that would permit specializations by the user.

[2017-06-10, Moved to Tentatively Ready after 6 positive votes on `c++std-lib`]

Proposed resolution:

This wording is relative to [N4659](#).

1. Change D.11 [depr.meta.types] as indicated:

-4- The behavior of a program that adds specializations for **any of the templates defined in this subclause `is_literal_type` or `is_literal_type_v`** is undefined, **unless explicitly permitted by the specification of the corresponding template.**

[2942](#). LWG 2873's resolution missed `weak_ptr::owner_before`

Section: 20.11.2.3.5 [util.smartptr.weak.obs] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2017-03-09 **Last modified:** 2017-06-15

Priority: Not Prioritized

View all other [issues](#) in [util.smartptr.weak.obs].

View all issues with [Tentatively Ready](#) status.

Discussion:

The NB comment asked for noexcept on shared_ptr::owner_before, weak_ptr::owner_before, and owner_less::operator(), but the PR of LWG [2873](#) only added it to the first and the third one.

[2017-06-03, Moved to Tentatively Ready after 6 positive votes on c++std-lib]

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 20.11.2.3 [util.smartptr.weak], class template weak_ptr synopsis, as indicated:

```
[...]
// 20.11.2.3.5 [util.smartptr.weak.obs], observers
[...]
template<class U> bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U> bool owner_before(const weak_ptr<U>& b) const noexcept;
```

2. Edit 20.11.2.3.5 [util.smartptr.weak.obs] immediately before p4 as indicated:

```
template<class U> bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U> bool owner_before(const weak_ptr<U>& b) const noexcept;
```

-4- *Returns:* An unspecified value such that [...]

[2954](#). Specialization of the convenience variable templates should be prohibited

Section: 17.5.4 [constraints] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2017-03-31 **Last modified:** 2017-06-15

Priority: Not Prioritized

View all issues with [Tentatively Ready](#) status.

Discussion:

There's currently no rule against specializing the various _v convenience variable templates outside of <type_traits>.

There should be; foo_v<T> should be always equal to foo<T>::value. The correct way to influence, say, is_placeholder_v<T> should be to specialize is_placeholder, not is_placeholder_v. Otherwise, the editorial changes to use the _v form to the specification would no longer be editorial but have normative impact.

Adding a global prohibition in 17.5.4.2.1 [namespace.std] seems preferable to adding individual prohibitions to each affected template; the PR below carves out an exception for variable templates that are intended to be specialized by users. As far as I know there are no such templates in the current WP, but the Ranges TS does use them.

[2017-06-14, Moved to Tentatively Ready after 6 positive votes on c++std-lib]

Proposed resolution:

This wording is relative to [N4659](#).

1. Add a paragraph to 17.5.4.2.1 [namespace.std], before p2:

-1- The behavior of a C++ program is undefined if it adds declarations or definitions to namespace std or to a namespace within namespace std unless otherwise specified. A program may add a template specialization for any standard library template to namespace std only if the declaration depends on a user-defined type and the specialization meets the standard library requirements for the original template and is not explicitly prohibited.(footnote: [...])

-?- The behavior of a C++ program is undefined if it declares an explicit or partial specialization of any standard library variable template, except where explicitly permitted by the specification of that variable template.

-2- The behavior of a C++ program is undefined if it declares [...]

2961. Bad postcondition for `set_default_resource`

Section: 20.12.4 [mem.res.global] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2017-05-09 **Last modified:** 2017-06-15

Priority: Not Prioritized

View all issues with [Tentatively Ready](#) status.

Discussion:

The specification of `set_default_resource` in [N4659](#) 20.12.4 [mem.res.global] reads:

```
memory_resource* set_default_resource(memory_resource* r) noexcept;
```

-4- *Effects:* If `r` is non-null, sets the value of the default memory resource pointer to `r`, otherwise sets the default memory resource pointer to `new_delete_resource()`.

-5- *Postconditions:* `get_default_resource() == r`.

-6- *Returns:* The previous value of the default memory resource pointer.

-7- *Remarks:* [...]

It is apparent that the effects specified in para 4 cannot — and indeed should not — achieve the postcondition specified in para 5 when `r` is null.

[2017-05-13, Tim comments]

This is the same issue as LWG [2522](#), which *just* missed the Fundamentals TS working draft used for the merge. A similar resolution (simply striking the *Postconditions*: paragraph) might be better.

Previous resolution [SUPERSEDED]:

This wording is relative to [N4659](#).

1. Modify 20.12.4 [mem.res.global] as follows:

```
memory_resource* set_default_resource(memory_resource* r) noexcept;
```

-4- *Effects:* If `r` is non-null, sets the value of the default memory resource pointer to `r`, otherwise sets the default memory resource pointer to `new_delete_resource()`.

-5- *Postconditions:* If `r` is non-null, `get_default_resource() == r`. Otherwise, `get_default_resource() == new_delete_resource()`.

-6- *Returns:* The previous value of the default memory resource pointer.

-7- *Remarks:* [...]

[2017-06-13, Casey Carter revises proposed wording]

I suggest to strike the *Postconditions* paragraph ([mem.res.global]/5) completely, as in LWG [2522](#)

[2017-06-15, Moved to Tentatively Ready after 6 positive votes on `c++std-lib`]

Proposed resolution:

This wording is relative to [N4659](#).

1. Modify 20.12.4 [mem.res.global] as follows:

```
memory_resource* set_default_resource(memory_resource* r) noexcept;
```

-4- *Effects*: If `r` is non-null, sets the value of the default memory resource pointer to `r`, otherwise sets the default memory resource pointer to `new_delete_resource()`.

~~-5- *Postconditions*: `get_default_resource() == r`.~~

-6- *Returns*: The previous value of the default memory resource pointer.

-7- *Remarks*: [...]

[2966](#). Incomplete resolution of US 74

Section: 99 [fs.path.native.obs] **Status**: [Tentatively Ready](#) **Submitter**: Jonathan Wakely **Opened**: 2017-05-25 **Last modified**: 2017-06-15

Priority: Not Prioritized

View all issues with [Tentatively Ready](#) status.

Discussion:

[P0492R1 US-74](#) failed to replace one occurrence of `pathstring`, in [fs.path.native.obs] p8. It should be changed to `native()`, as with the other native format observers.

[2017-06-12, Moved to Tentatively Ready after 6 positive votes on c++std-lib]

Proposed resolution:

This wording is relative to [N4659](#).

1. Edit 99 [fs.path.native.obs] as indicated:

```
std::string string() const;
std::wstring wstring() const;
std::string u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;
```

-8- *Returns*: ~~`pathstring`~~`native()`.

-9- *Remarks*: Conversion, if any, is performed as specified by [fs.path.cvt]. The encoding of the string returned by `u8string()` is always UTF-8.

[2974](#). Diagnose out of bounds `tuple_element/variant_alternative`

Section: 20.4.4 [pair.astuple], 20.7.4 [variant.helper] **Status**: [Tentatively Ready](#) **Submitter**: Agustín K-ballo Bergé **Opened**: 2017-06-10 **Last modified**: 2017-06-15

Priority: Not Prioritized

View all other issues in [pair.astuple].

View all issues with [Tentatively Ready](#) status.

Discussion:

Instantiating `tuple_element` with an out-of-bounds index requires a diagnostic for `tuple` and `array`, but not for `pair`. The specification requires an out-of-bounds index for `pair` to go to the base template instead, which is undefined.

Similarly, instantiating `variant_alternative` with an out-of-bounds index violates a requirement, but it is not required to be diagnosed.

[2017-06-12, Moved to Tentatively Ready after 5 positive votes on c++std-lib]

Proposed resolution:

This wording is relative to [N4659](#).

1. Modify 20.2.1 [utility.syn], header `<utility>` synopsis, as indicated:

[...]

```
// 20.4.4 [pair.astuple], tuple-like access to pair
template <class T> class tuple_size;
template <size_t I, class T> class tuple_element;
template <class T1, class T2> struct tuple_size<pair<T1, T2>>;
template <size_t I, class T1, class T2> struct tuple_element<I, pair<T1, T2>>;
template <class T1, class T2> struct tuple_element<0, pair<T1, T2>>;
template <class T1, class T2> struct tuple_element<1, pair<T1, T2>>;
```

[...]

2. Modify 20.4.4 [pair.astuple] as indicated:

```
tuple_element<0, pair<T1, T2>>::type
```

~~-1- Value: The type T_1 .~~

```
tuple_element<1, pair<T1, T2>>::type
```

~~-2- Value: The type T_2 .~~

```
tuple_element<I, pair<T1, T2>>::type
```

~~-?- Requires: $I < 2$. The program is ill-formed if I is out of bounds.~~

~~-?- Value: The type T_1 if $I == 0$, otherwise the type T_2 .~~

3. Modify 20.7.4 [variant.helper] as indicated:

```
variant_alternative<I, variant<Types...>>::type
```

~~-4- Requires: $I < \text{sizeof}...(Types)$. The program is ill-formed if I is out of bounds.~~

~~-5- Value: The type T_I .~~